

Roll No: CS25M052
Collaborators (if any):

Name: Tushar

1. Part 1: Sentence Segmentation

1. Suggest a simplistic top-down approach to sentence segmentation for English texts. Do you foresee issues with your proposed approach in specific situations? Provide supporting examples and possible strategies that can be adopted to handle these issues. [2 marks]

NLTK is one of the most commonly used packages for Natural Language Processing. What does the Punkt Sentence Segmenter in NLTK do differently from a simple top-down (rule-based) sentence segmentation approach? Additionally, compare both of the above approaches with spaCy's sentence segmentation method. [1 mark]

2. Design an adversarial test suite for sentence segmentation: [4 marks]

(a) Create 15 sentences that are likely to break a naive segmentation system.

(b) Categorize them by error type (e.g., abbreviations, decimals, ellipsis, quotes, multiple punctuation, etc.).

(c) Evaluate how many errors each segmenter makes on your test suite.

Solution:

1. A simple top-down approach splits text into sentences using punctuation marks such as: . ! ?

Algorithm:-

1. Scan the text from left to right.
2. Whenever any punctuation mark is encountered, split the text.
3. The string before the punctuation mark is treated as a single sentence.

Example:

Text: MS Dhoni finishes off in style! MS Dhoni is the only captain in international cricket history to win all three major ICC trophies.

Output:

Sentence1: MS Dhoni finishes off in style

Sentence2: MS Dhoni is the only captain in international cricket history to win all three major ICC trophies

Problems with this approach:-

1. ABBREVIATIONS:

Text: Dr. Shruti is a professor.

Output:

Sentence1: Dr

Sentence2: Shruti is a professor

2. DECIMAL NUMBERS:

Text: The value of e(Euler's number) is 2.718.

Output:

Sentence1: The value of e(Euler's number) is 2

Sentence2: 718

3. NUMERICAL LISTS:

Text: 1. Prepare the data. 2. Train the model.

Output:

Sentence1: 1

Sentence2: Prepare the data

Sentence3: 2

Sentence4: Train the model

4. WEBISTES AND EMAILS:

Text: Email me at tushar@smail.iitm.ac.in for details.

Output:

Sentence1: Email me at tushar@smail

Sentence2: iitm

Sentence3: ac

Sentence4: in for details

Possible Strategies:-

1. **Abbreviations:** Maintain a pre-defined list of common abbreviations (e.g., Mr., Dr., Inc., Ltd.). If a period is preceded by a word in this list, the system is instructed not to split.
2. **Numerical Decimals:** If the period is surrounded by digits on both sides (e.g., 3.14), it is identified as a decimal point rather than a sentence termination.
3. **Enumeration Patterns:** Identify patterns such as (Digit + Period + Space + Capital). This ensures the period is treated as a list enumerator rather than a terminal sentence marker.

4. **Machine Learning:** Implement machine learning-based segmentation for more complex problems.

Table 1: Characteristics of Different Sentence Segmentation Methods

Aspect	Rule-Based Method	NLTK Punkt Tokenizer	spaCy Sentence Seg- menter
Underlying Idea	Relies on manually specified punctuation rules to detect sentence boundaries.	Uses an unsupervised statistical algorithm .	Uses linguistic information from a pretrained NLP pipeline.
Use Cases	Suitable for simple pre-processing tasks.	Useful for general text processing where moderate accuracy is sufficient.	Preferred in full NLP pipelines and real-world applications requiring strong accuracy.

Table 2: Categorization of Test Suite by Error Type (2.a and 2.b)

No.	Sentence for Testing	Challenge Type
1	M.S. Dhoni led CSK to multiple IPL titles with his "cool" captaincy.	Abbreviation
2	I submitted my NLP assignment at 11:59 p.m. sharp.	Time abbreviation
3	"Will India win the next World Cup?" asked the nervous cricket fan.	Quotes + question mark
4	The algorithm's accuracy is 98.5% on this dataset.	Decimal
5	1. Start the simulation. 2. Analyze the loss function.	Numbered list
6	He said, "Dhoni is the best finisher!" The whole stadium cheered.	Quote boundary
7	Visit www.iitm.ac.in to check the M.Tech admission results for 2026.	URL + abbreviation
8	What?! Jadeja hit a six on the last ball!! Unbelievable!	Multiple punctuation
9	Dr. APJ Abdul Kalam remains an inspiration for every Indian student.	Title abbreviation
10	The match starts in 15 minutes...please hurry up to the MA Chidambaram Stadium.	Ellipsis
11	Contact the HOD at cseoffice@institute.ac.in for elective queries.	Email address
12	The U.S. and India are collaborating on advanced AI research projects.	Abbreviation
13	"Wait!" the umpire shouted, "The bowler has overstepped the line!"	Multiple quotes
14	The price of the signed cricket bat was 75,000.50 at the auction.	Decimal
15	Prof. Sutanu's lecture on NLP was great...looking forward to the next one.	Title + ellipsis

2. c)

Table 3: Sentence Segmentation Errors on the Adversarial Test Suite

Method	Errors (out of 15)
Naive Rule-Based	12
NLTK Punkt	2
spaCy	0

```
Sentence 1
M.S. Dhoni led CSK to multiple IPL titles with his cool captaincy.

Naive:
['M', 'S', 'Dhoni led CSK to multiple IPL titles with his cool captaincy']

Punkt:
['M.S.', 'Dhoni led CSK to multiple IPL titles with his cool captaincy.']

spaCy:
['M.S. Dhoni led CSK to multiple IPL titles with his cool captaincy.']
```

Figure 1: Example

2. Part 2: Tokenization

1. Suggest a simplistic top-down approach for tokenization in English text. Identify specific situations where your proposed approach may fail to produce expected results. [1 mark]
2. Study about NLTK's Penn Treebank tokenizer and spaCy's tokenizer. What type of knowledge does each of them use — Top-down, Bottomup, or a hybrid approach? Clearly justify your answer for both tokenizers.[2 marks]
3. Perform word tokenization of the sentence-segmented documents using: [4 marks]
 - (a) The proposed top-down method
 - (b) Penn Treebank Tokenizer
 - (c) spaCy Tokenizer

State a possible scenario along with an example where

- (a) Your approach performs better than the Penn Treebank and spaCy tokenizers
- (b) Your approach performs worse than the Penn Treebank and spaCy tokenizers

Solution:

1. The naive tokenizer uses simple regular expressions to split text into tokens based on alphanumeric word boundaries. Tokens are extracted using a regular expression that matches sequences of alphanumeric characters.

Algorithm:-

1. Take the segmented sentence as input.
2. Scan the sentence from left to right.
3. Identify tokens using a regular expression that extracts sequences of alphanumeric characters (e.g., `\b\w+\b`).
4. Ignore punctuation marks and special characters while extracting tokens.
5. Store the extracted tokens in a list corresponding to the sentence.

Example:

Sentence: The aircraft flies at high altitude.

Naive tokens: ["The", "aircraft", "flies", "at", "high", "altitude"]

Problems with the Naive Tokenization Approach:**• Contractions:**

Contractions contain apostrophes(') that represent shortened word forms. Naive tokenization splits them incorrectly.

Example:

Sentence: Don't split contractions incorrectly.

Naive Tokens: ["Don", "t", "split", "contractions", "incorrectly"]

• Hyphenated Words:

Hyphenated expressions represent a single concept, but naive tokenization separates them into multiple tokens.

Example:

Sentence: The experiment used a long-term evaluation strategy.

Naive Tokens: ["The", "experiment", "used", "a", "long", "term", "evaluation", "strategy"]

• Special Characters and Expressions:

Tokens containing special characters such as email addresses or URLs are broken into multiple parts.

Example:

Sentence: Contact us at tushar@iitm.com for assistance.

Naive Tokens: ["Contact", "us", "at", "support", "example", "com", "for", "assistance"]

2.

NLTK Penn Treebank Tokenizer:

The Penn Treebank tokenizer follows a primarily top-down rule-based approach. It uses a pre-defined set of linguistic rules designed to mimic the tokenization conventions used in the Penn Treebank corpus. These rules explicitly specify how punctuation, contractions, and special cases should be split.

spaCy Tokenizer:

The spaCy tokenizer follows a hybrid approach that combines both top-down and bottom-up strategies. It first applies a set of language-specific rules and exception lists (top-down knowledge) to identify special cases such as contractions, abbreviations, URLs, and email addresses. At the same time, spaCy also performs character-level analysis and iterative splitting of tokens based on prefixes, suffixes, and infixes (bottom-up processing).

Tokenizer	Approach	Reason
Penn Treebank (NLTK)	Top-down	Uses predefined rule-based patterns and regular expressions to split tokens
spaCy Tokenizer	Hybrid	Combines rule-based tokenization, exception rules, and statistical linguistic knowledge

3. a) Scenario where the proposed approach performs better than Penn Treebank and spaCy:-
The proposed tokenizer performs better in cases where punctuation marks appear inside tokens but should not be treated as meaningful linguistic elements. Since the naive tokenizer removes punctuation and extracts only alphanumeric sequences, it can produce cleaner tokens for certain information retrieval tasks.

Example:

Sentence:- Students enrolled in course CS-6190 must submit the final project by Friday.

Naive Tokenizer Output: ["Students", "enrolled", "in", "course", "CS", "6190", "must", "submit", "the", "final", "project", "by", "Friday"]

Penn Treebank Tokenizer Output: ["Students", "enrolled", "in", "course", "CS-6190", "must", "submit", "the", "final", "project", "by", "Friday", "."]

spaCy Tokenizer Output: ["Students", "enrolled", "in", "course", "CS-6190", "must", "submit", "the", "final", "project", "by", "Friday", "."]

3. b) Scenario where the proposed approach performs worse than Penn Treebank and spaCy:-
The naive tokenizer performs poorly when punctuation is part of the token itself and carries important semantic meaning. In such cases, removing punctuation or splitting tokens incorrectly can lead to loss of information.

Example:

Sentence:- The algorithm's accuracy is 98.5% on this dataset.

Naive Output: ["The", "algorithm", "s", "accuracy", "is", "98", "5", "on", "this", "dataset"]

Penn Treebank Output: ["The", "algorithm", "'s", "accuracy", "is", "98.5", "%", "on", "this", "dataset", "."]

spaCy Output: ["The", "algorithm", "'s", "accuracy", "is", "98.5", "%", "on", "this", "dataset", "."]

3. Part 3: Stemming and Lemmatization

1. What is the difference between stemming and lemmatization? Clearly explain the conceptual differences between the two approaches in terms of linguistic knowledge, output form, and computational complexity. Give suitable examples to illustrate your answer. [1 mark]

2. Using the word-tokenized text from the Cranfield Dataset: [3 marks]

(a) Perform stemming using Porter's Stemmer

(b) Perform lemmatization using WordNet Lemmatizer

Compare the outputs of the two methods by analyzing:

- Changes in vocabulary size
- Examples of over-stemming (if any)
- Cases where lemmatization preserves semantic meaning better

Solution:

Table 4: Comparison between Stemming and Lemmatization

Aspect	Stemming	Lemmatization
Linguistic Knowledge	Uses simple heuristic rules to remove suffixes without relying on linguistic knowledge or dictionaries.	Uses linguistic knowledge and lexical resources (e.g., WordNet) to determine the correct base form of a word.
Output Form	Produces a stem, which may not be a valid dictionary word.	Produces a lemma, which is usually a valid dictionary word.
Computational Complexity	Computationally simple and fast because it relies only on rule-based suffix removal.	Computationally more expensive because it involves dictionary lookup and sometimes grammatical analysis.
Example	running → run studies → studi	running → run studies → study

2. Changes in Vocabulary Size:-

Stemming aggressively reduces words by removing suffixes such as -ing, -ed, and -tion. As a result, many different word forms collapse into the same stem, which reduces the overall vocabulary size. Lemmatization, on the other hand, converts words to their correct dictionary base forms using linguistic knowledge and lexical resources. Because it preserves valid root words, the resulting vocabulary size is generally larger than that produced by stemming.

Table 5: Effect of Stemming and Lemmatization on Vocabulary Size

Method	Vocabulary Size	Reason
Porter Stemmer	7127 (Smaller)	Many words collapse into the same stem due to aggressive suffix removal.
WordNet Lemmatizer	8954 (Larger)	Words are converted into valid dictionary lemmas, preserving distinctions between related terms.

Example:-

Original Words: investigate, investigation, investigated

Stemming Output: investig, investig, investig

Lemmatization Output: investigate, investigation, investigate

Examples of Over-Stemming:=

Over-stemming occurs when different words are reduced to the same stem, which can lead to loss of semantic information.

Example:

Original Words: velocity, velocities

Stemming Output: veloc

Lemmatization Output: velocity

Cases Where Lemmatization Preserves Semantic Meaning Better:

Lemmatization preserves linguistic correctness by mapping words to valid base forms.

Example:

Original Word: studies

Stemming Output: studi

Lemmatization Output: study

Original Word: boundaries

Stemming Output: boundari

Lemmatization Output: boundary

4. Part 4: Stopword Removal

1. Remove stopwords from the tokenized documents using a curated list, such as the stopwords list provided by NLTK. [1 mark]
2. Suggest a bottom-up (data-driven) approach for constructing a stopwords list that is specific to the given corpus of documents. Clearly explain the intuition behind your proposed method. [1 mark]
3. Implement the strategy proposed in the previous question and compare the stopwords obtained with those provided by NLTK on the Cranfield dataset. Your comparison should include: [2 marks]
 - Number of stopwords identified
 - Overlap between the two lists
 - Words that appear in one list but not the other

Solution:**1. Example:****Original tokens:** ['the', 'flow', 'field', 'in', 'a', 'radial', 'compressor']**After stopword removal:** ['flow', 'field', 'radial', 'compressor']**2. Bottom-Up Stopword Construction (Data-Driven Approach)**

Instead of relying on a predefined stopword list, a stopword list can also be constructed using a data-driven approach. The intuition behind this method is that words that occur extremely frequently across many documents are often less informative for distinguishing between documents.

The bottom-up strategy works as follows:

- Compute the frequency of every word in the corpus.
- Identify words that appear very frequently across the dataset.
- Treat these high-frequency words as candidate stopwords.
- Construct a stopword list based on these frequently occurring terms.

3. Comparison between NLTK Stopwords and Corpus-Derived Stopwords:

The stopwords obtained using the NLTK list were compared with the stopwords obtained using the corpus-driven frequency-based approach.

Table 6: Comparison of Stopword Lists (Part 1)

Metric	NLTK Stopwords	Corpus-Derived Stopwords
Number of Stopwords	198 words in the pre-defined English stopword list provided by NLTK	25 words identified using frequency analysis on the Cranfield corpus
Overlap	20 words from the NLTK list also appear in the corpus-derived stopword list	These overlapping words correspond mainly to common function words frequently occurring in the dataset

Table 7: Comparison of Stopword Lists (Part 2)

Metric	NLTK Stopwords	Corpus-Derived Stopwords
Words only in NLTK	Examples include isn, wouldn, but, down, off which are common English function words	These words appear less frequently in the Cranfield technical documents and therefore were not selected by the frequency-based approach
Words only in Corpus List	These words are not included in the NLTK stopwords list because they carry semantic meaning in general English	Examples include flow, pressure which appear very frequently in the aerodynamics documents of the Cranfield dataset

```
NLTK Stopwords: 198
Corpus Stopwords: 25
Overlap: 20
```

Figure 2: Result by taking threshold as 0.004

5. Part 5: Retrieval

1. Given a set of queries Q and a corpus of documents D , what would be the total number of similarity computations required to estimate the similarity between every query and every document?

Assume that TF-IDF vector representations are already available for both queries and documents over a vocabulary V . Clearly express your answer in terms of $|Q|$, $|D|$, and $|V|$. [1 mark]

2. Suggest how the concept of an inverted index can be used to reduce the time complexity derived in the previous question. You may introduce additional variables (such as average document frequency or sparsity factors) to explain your reasoning. Provide a clear comparison between the naive approach and the inverted-index-based approach in terms of computational complexity. [2 marks]

Solution:

1. Total Number of Similarity Computations

Let:

- $|Q|$ denote the number of queries
- $|D|$ denote the number of documents in the corpus
- $|V|$ denote the vocabulary size

Each query and document is represented as a TF-IDF vector over the vocabulary V . To compute the similarity between a query and a document, a common approach is to use cosine similarity between their TF-IDF vectors. The cosine similarity between a query vector q and a document vector d is computed as:

$$\text{sim}(q, d) = \frac{q \cdot d}{||q|| ||d||}$$

Computing the dot product requires iterating over all vocabulary dimensions. Therefore, the time complexity of computing similarity between a single query-document pair is:

$$O(|V|)$$

Since the similarity must be computed for every query-document pair, the total number of computations required is:

$$|Q| \times |D|$$

Thus, the overall computational complexity becomes:

$$O(|Q| \times |D| \times |V|)$$

2. Improving Efficiency with an Inverted Index

An inverted index associates each term in the vocabulary with the list of documents in which it appears. Using this, the system avoids comparing a query with every document and instead considers only documents that contain at least one query term. Let:

- $df(t)$ be the number of documents containing term t
- a be the average number of terms in a query

If $\hat{df}$ represents the average document frequency of query terms, the number of candidate documents can be approximated as: $a \times \hat{df}$. Thus, the overall complexity using an inverted index becomes:

$$O(|Q| \times a \times \hat{df})$$

Since $a \ll |V|$ and $\hat{df} \ll |D|$, this method is significantly more efficient than the naive approach.

Comparison

Method	Complexity
Naive retrieval	$O(Q \times D \times V)$
Inverted index retrieval	$O(Q \times a \times \hat{df})$